



Light Hypergraph Models

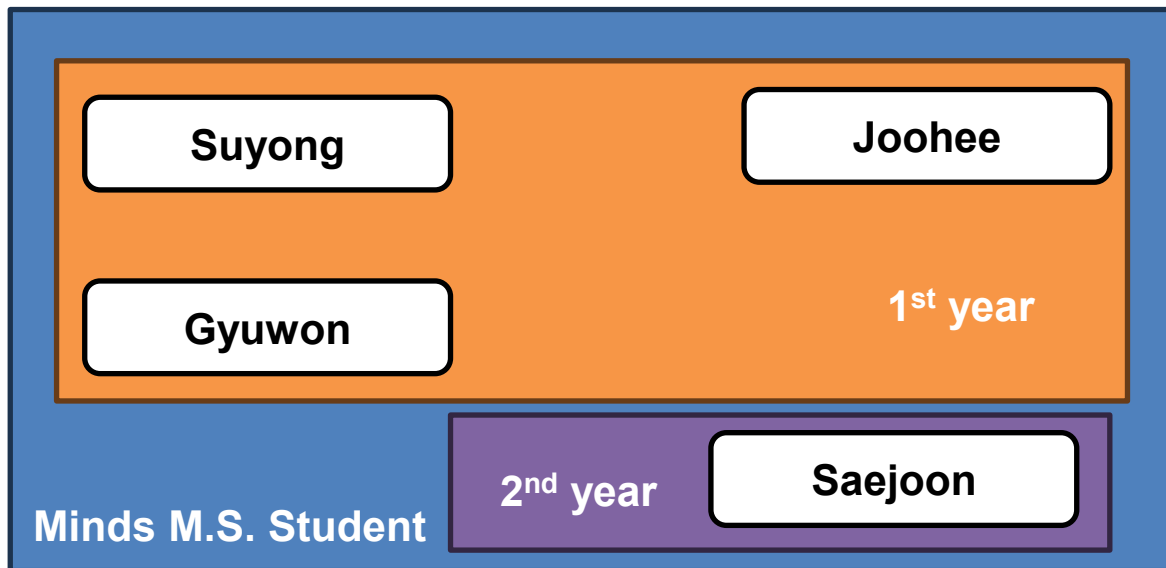
2026.03.24

Joohee Cho

Machine Intelligence & Data Science (MINDS) Lab
Chung-Ang University

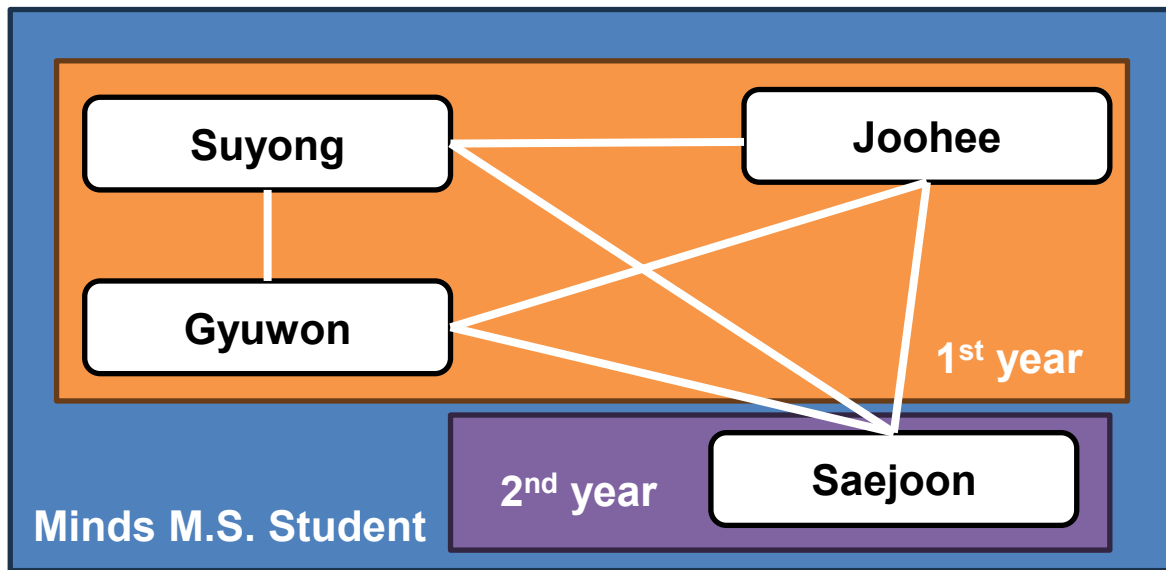
High-Order Relation

- There are many relations that cannot be represented as simple bi-partite formats in real world



High-Order Relation (Cont.)

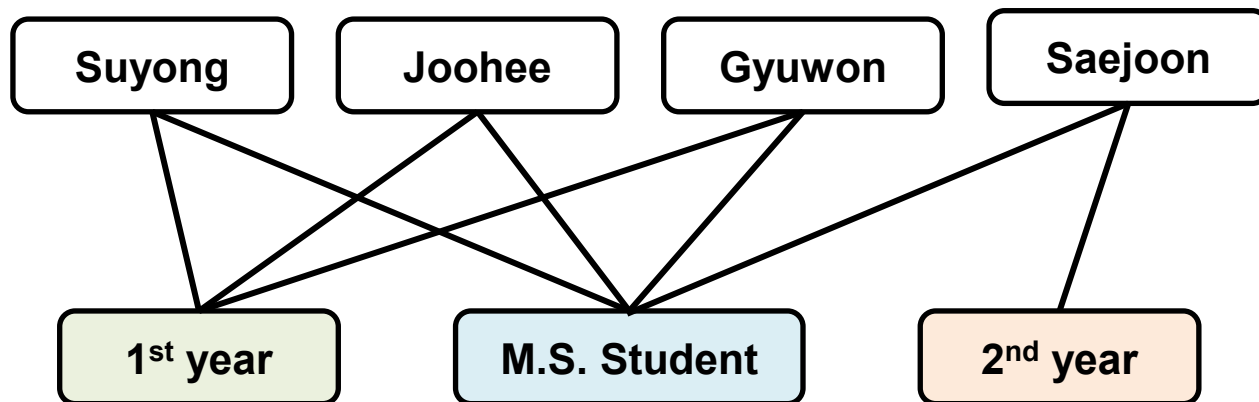
- Graphs loses information of sets; therefore, we need a new method to preserve the relational information



Hypergraph Neural Network

- Hypergraph uses sets (hyperedges) to represent high-order relationships between the nodes (hypernodes)

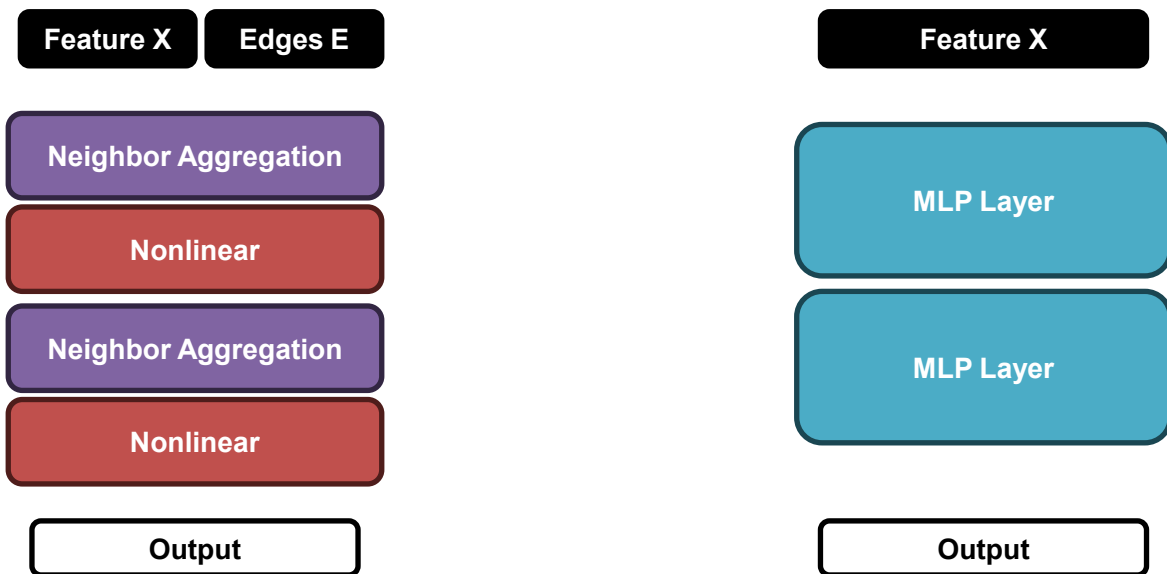
■ V : Set of hypernodes, E : Set of hyperedges



H	1 st year	M.S. Student	2 nd year
Suyong	1	1	0
Joohee	1	1	0
Gyuwon	1	1	0
Saejoon	0	1	1

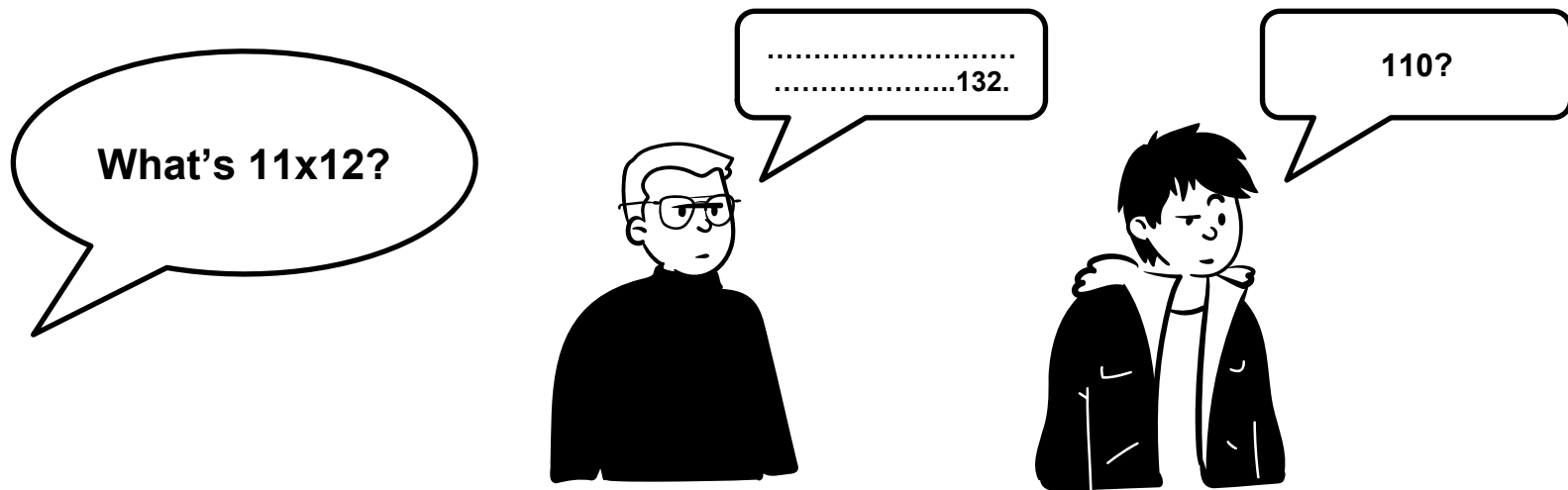
Problem of Graphs

- Utilizing graph requires more computation, thus requires longer inference time and computing resource

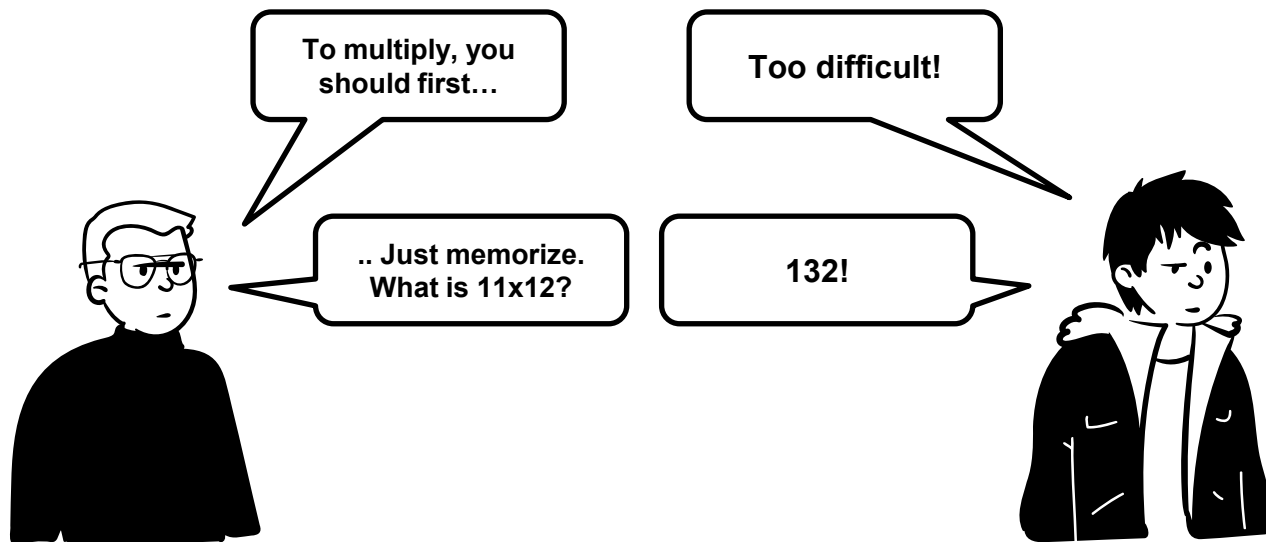


How to Solve It?

- Big models are accurate but slow, and small models are usually fast but inaccurate
 - Is accurate and fast model possible?

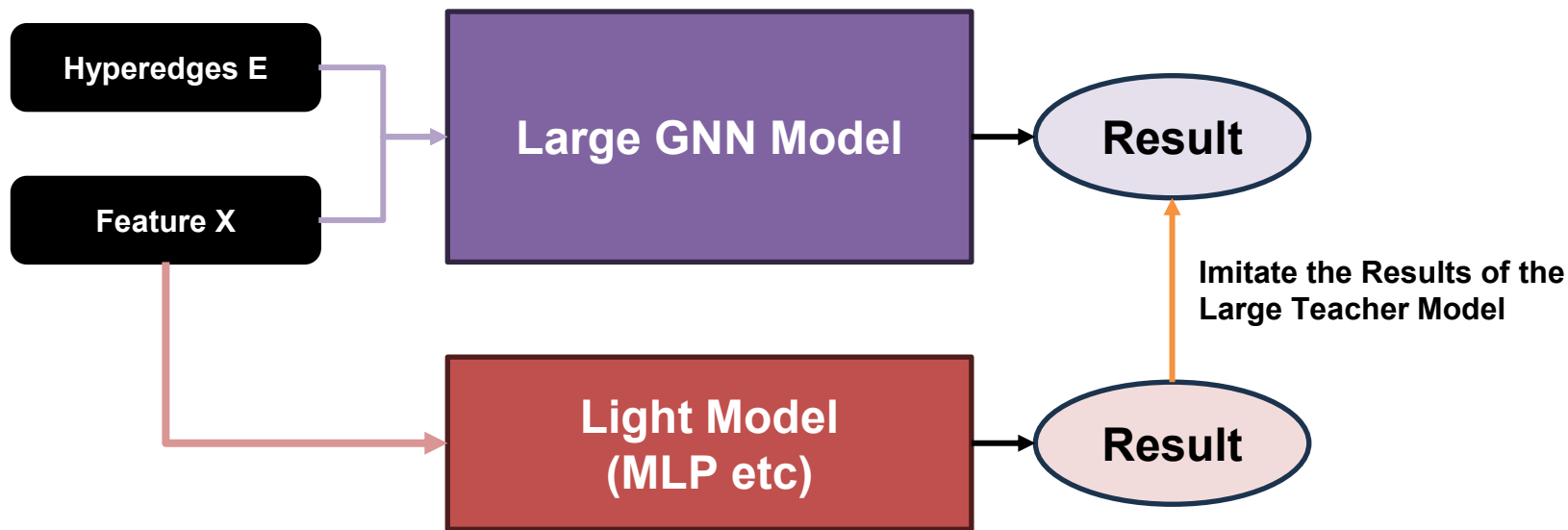


- Big teacher models teach knowledge to small student models to make their accuracy increase while maintaining speed and computing power



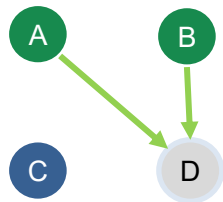
Distillation Types – Logits

- Student model learns the result values of teacher models

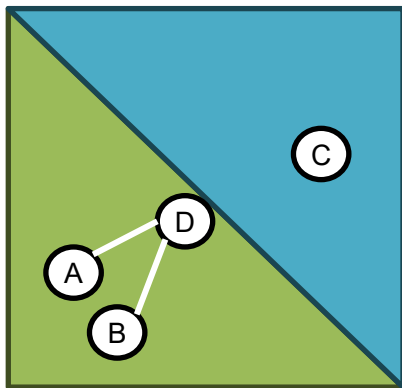


Distillation Types – Embeddings

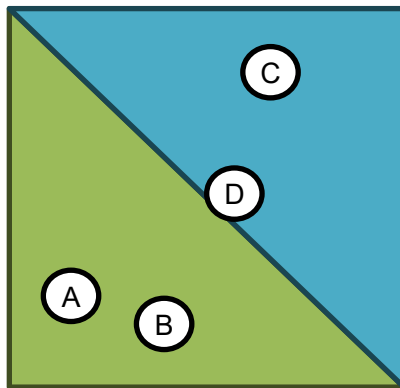
- Imitate the process teacher model learned



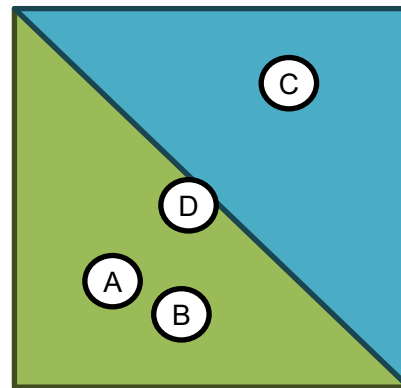
Label	Nodes
Green	A, B, D
Blue	C



Teacher GNN Model



Logit Distilled Model

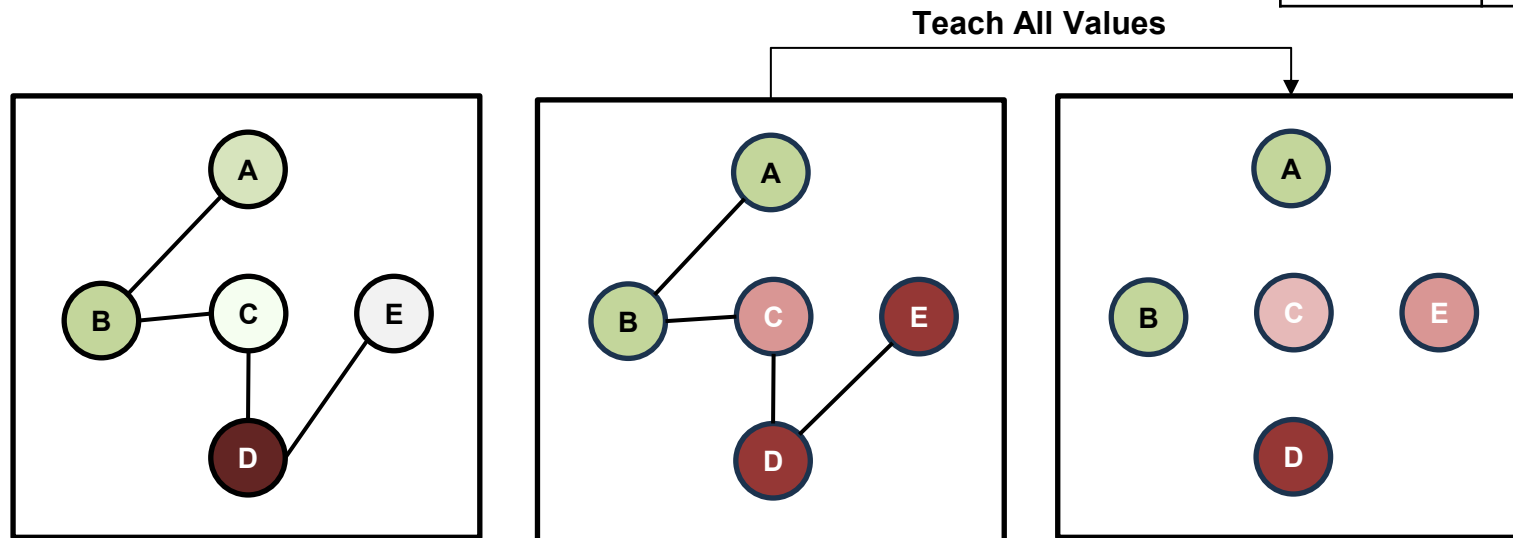


Logit + Embedding Distilled Model

Distillation Types – Direct

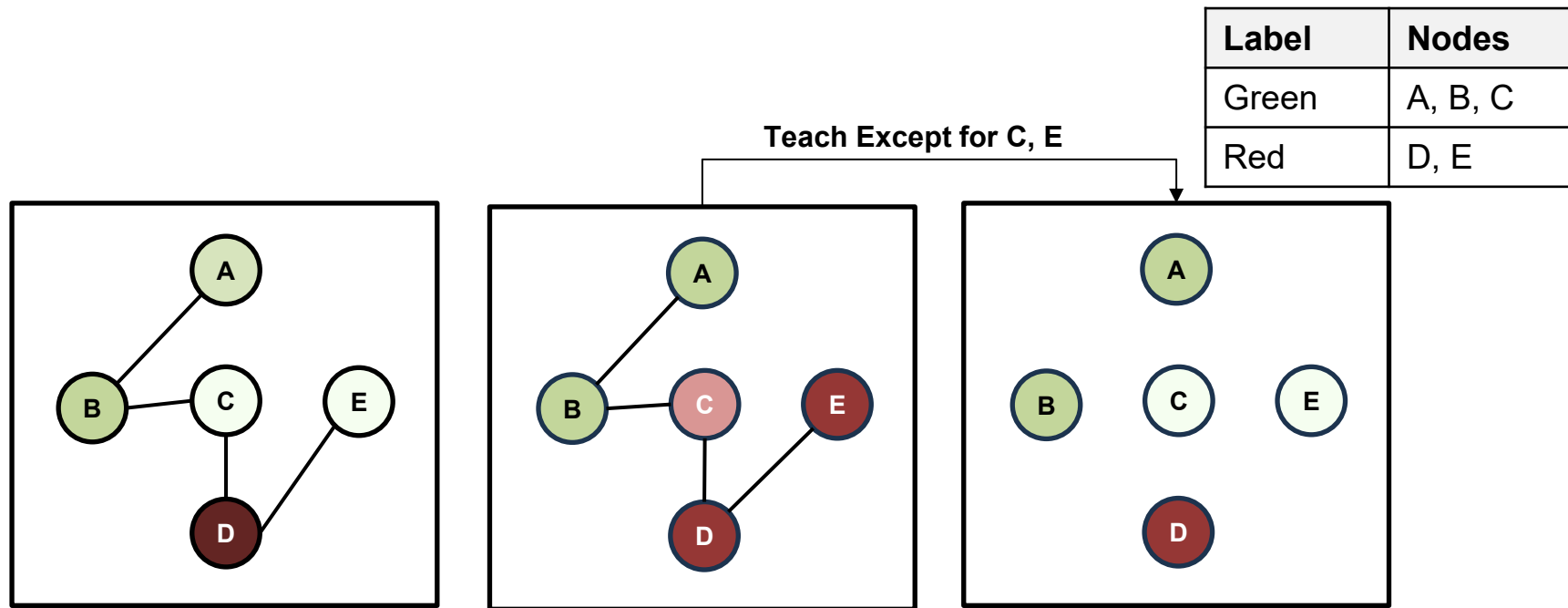
- Teach all values of the teacher to the student model

Label	Nodes
Green	A, B, C
Red	D, E



Distillation Types – Adaptive

- Selectively teach values of the teacher to the student model





Adaptive and Reinforcement-Guided Contrastive Hypergraph Distillation

WSDM '26

Rongwei Xu, Zitai Qiu, Pengfei Ding, Yan Wang,
Jia Wu, Amin Beheshti

Macquarie University

□ Create graphs for each hyperedges and apply GCN to the graphs

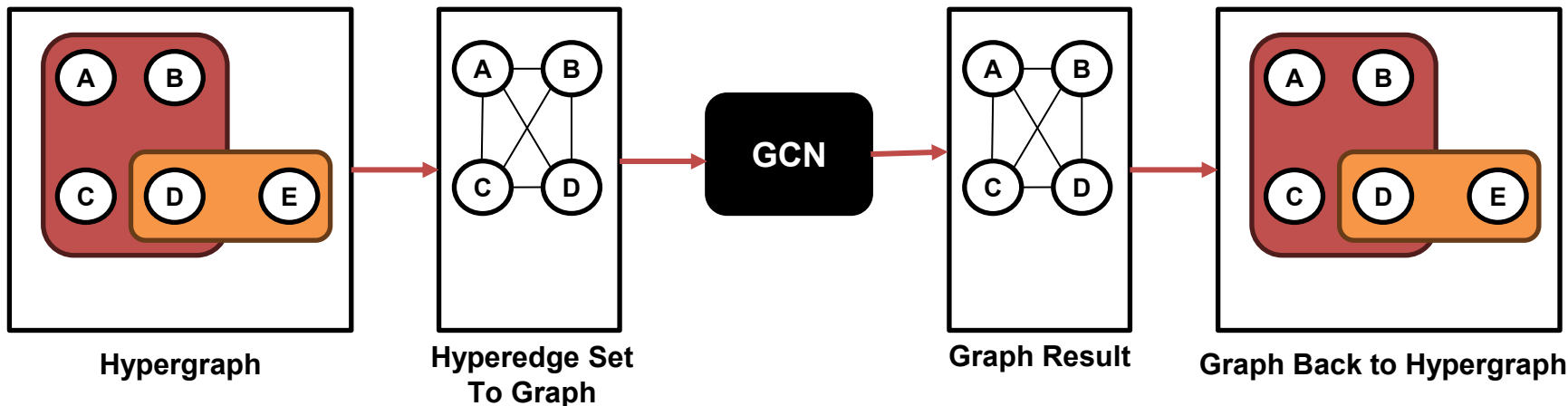
■ HGNN is spectral based, and UniGCN is special based

$$\mathbf{Y} = \mathbf{D}_v^{-1/2} \mathbf{H} \mathbf{W} \mathbf{D}_e^{-1} \mathbf{H}^T \mathbf{D}_v^{-1/2} \mathbf{X} \Theta$$

HGNN

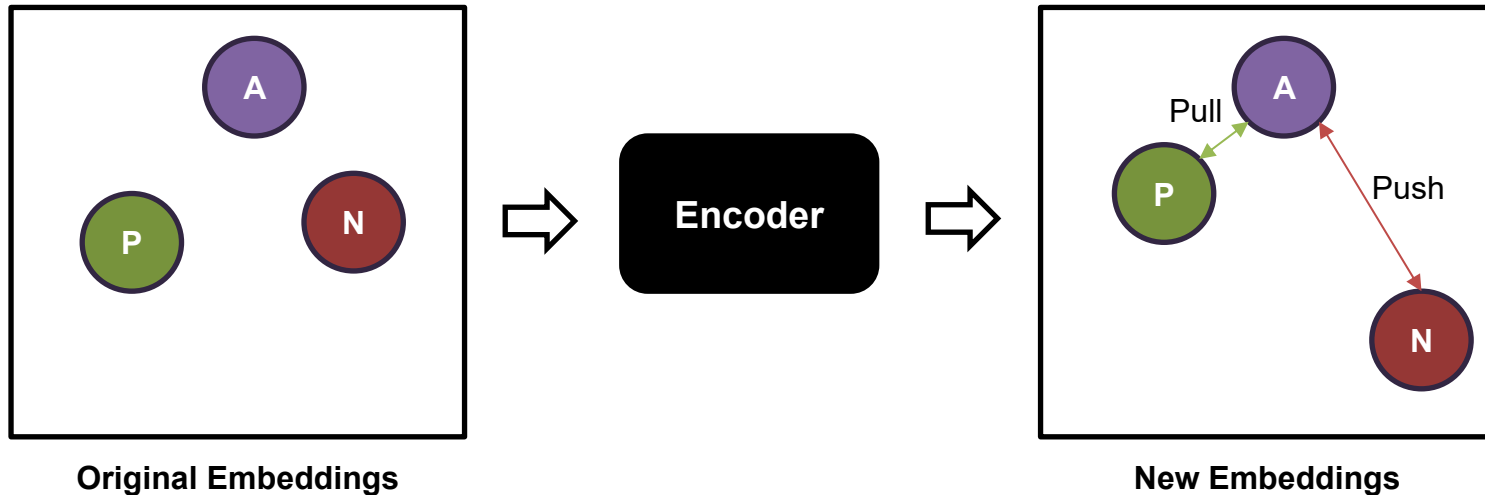
$$(\text{UniGCN}) \begin{cases} h_e = \phi_1(\{x_j\}_{j \in e}) \\ \tilde{x}_i = \phi_2(x_i, \{h_e\}_{e \in E_i}) \end{cases}$$

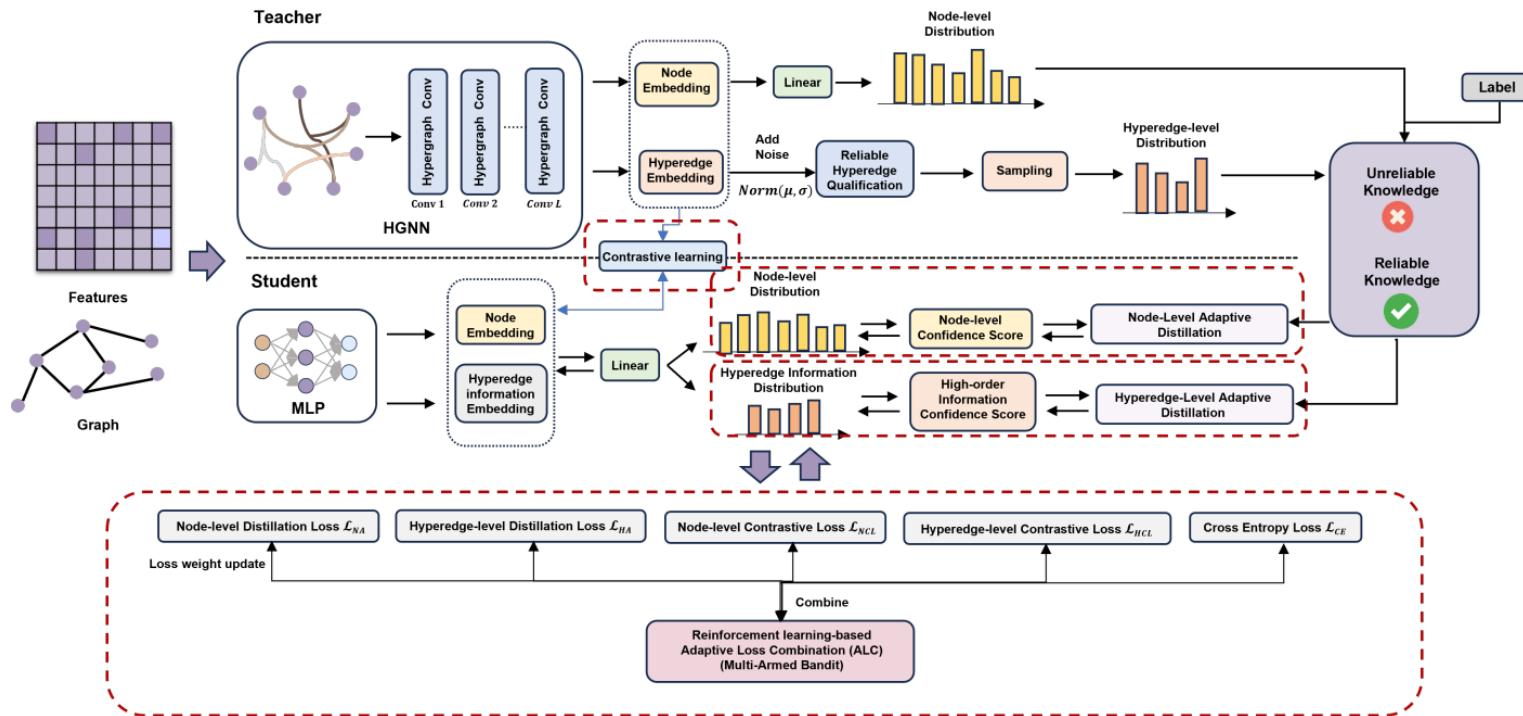
$$\tilde{x}_i = \frac{1}{\sqrt{d_i}} \sum_{e \in \tilde{E}_i} \frac{1}{\sqrt{d_e}} W h_e,$$



Contrastive Learning

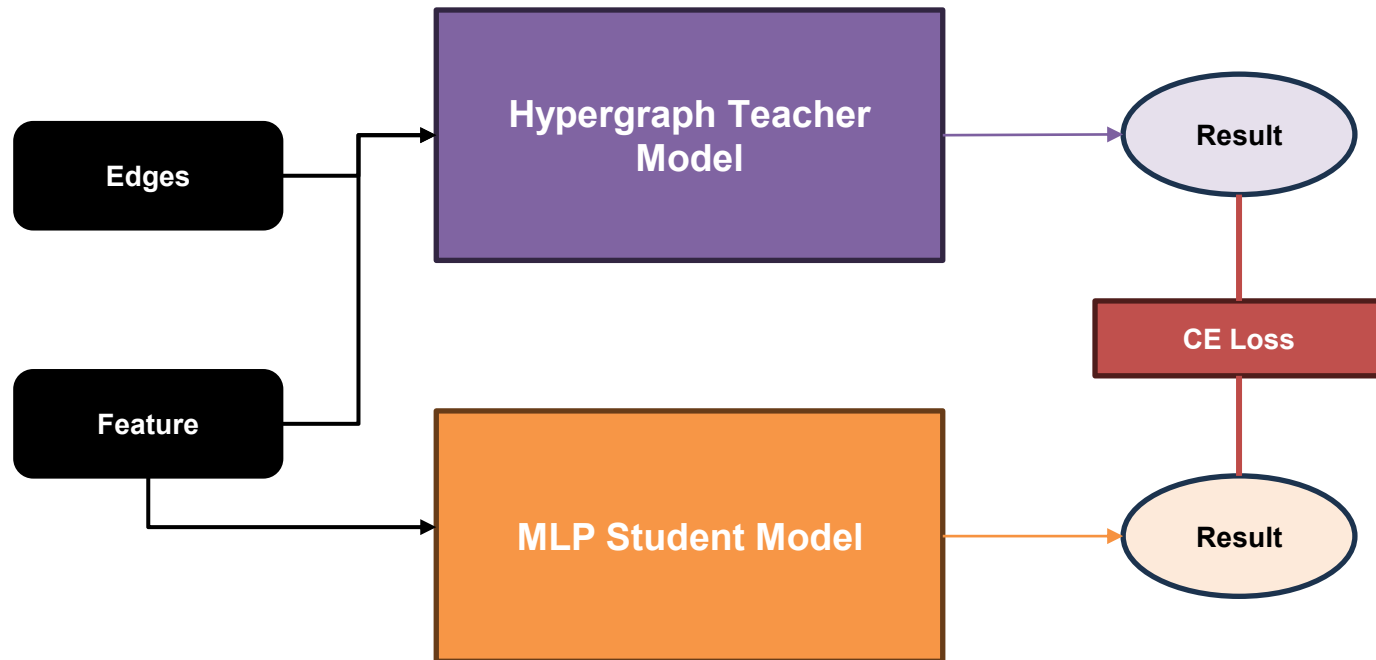
- In the embedding space, pull positive nodes closer and push negative nodes further from the anchor





Cross Entropy Loss

- Logit distillation between hypergraph teacher and MLP student model



Temperature

- Apply temperature based on the confidence of student and teacher accuracy

■ z : embeddings, T : temperature, p : soft labels, $y'(v)$: teacher prediction, $y(v)$: true label

$$p_v^t = \text{softmax}(z_v^t/T), \quad p_v^s = \text{softmax}(z_v^s/T)$$

$$\phi(z_v^s, z_v^t, y(v)) = \begin{cases} T_{\max}^{\text{node}}, & \text{if } y'(v) \neq y(v) \\ \tilde{T}(v), & \text{otherwise} \end{cases},$$

$$\tilde{T}(v) = T_{\max}^{\text{node}} - (T_{\max}^{\text{node}} - T_{\min}^{\text{node}}) \cdot \max(z_v^s)$$

T high :

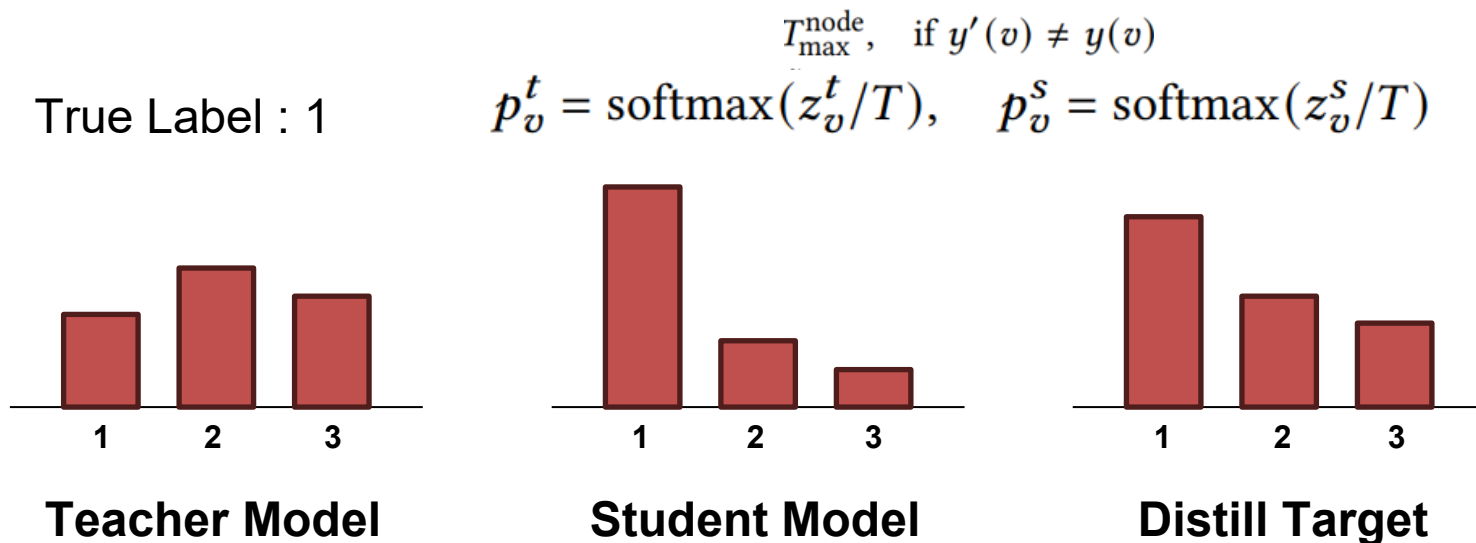
Reduce Influence of Teacher

T Low :

Increase Influence of Teacher

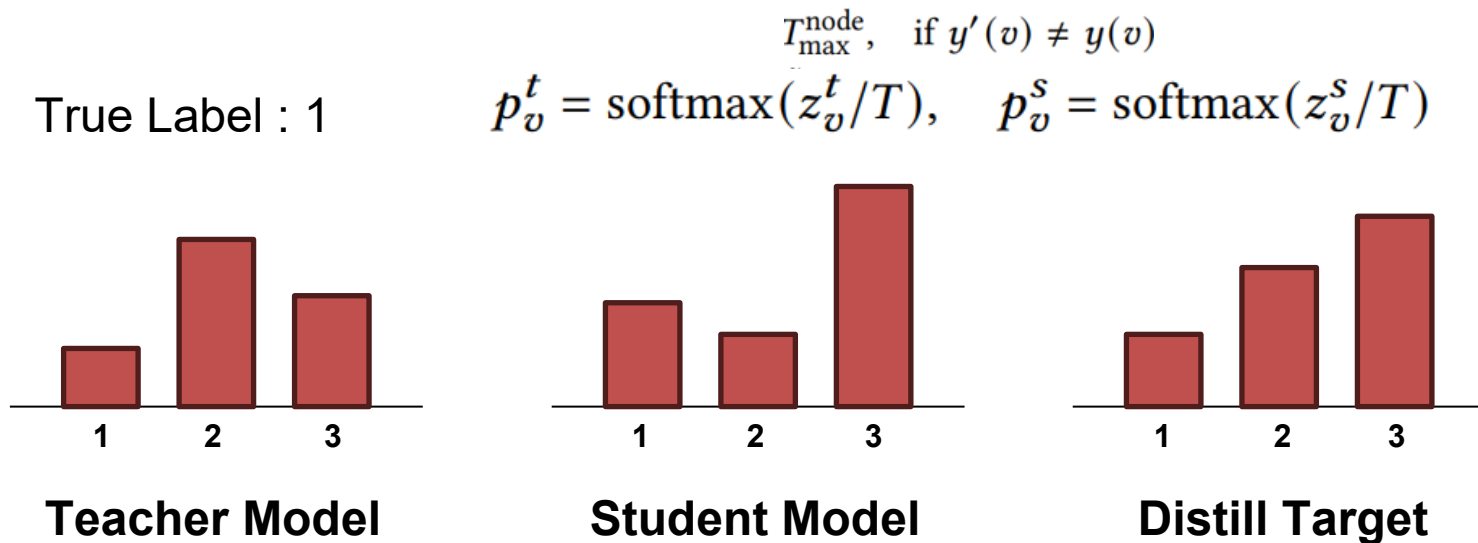
Temperature – $y'(v) \neq y(v)$

- If true label is different from teacher model's value, preserve student model's value, setting T to max, preserving student more



Temperature – $y'(v) \neq y(v)$ (Cont.)

- If the student model is also wrong, it might still not do its role

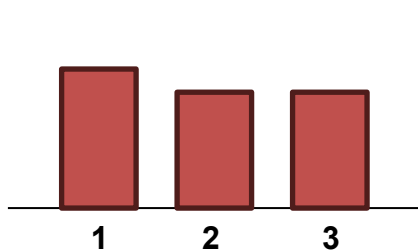


Temperature – $y'(v) = y(v)$, S Confident

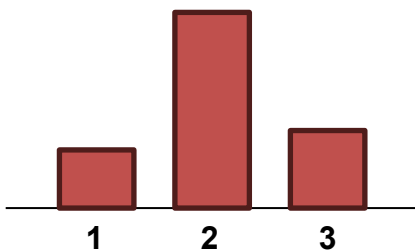
- If true label and the teacher predicted label is equal and the student is confident with its value, T gets smaller, transferring more

True Label : 1

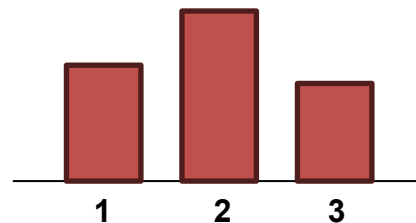
$$\tilde{T}(v) = T_{\max}^{\text{node}} - (T_{\max}^{\text{node}} - T_{\min}^{\text{node}}) \cdot \max(z_v^s)$$
$$p_v^t = \text{softmax}(z_v^t/T), \quad p_v^s = \text{softmax}(z_v^s/T)$$



Teacher Model



Student Model



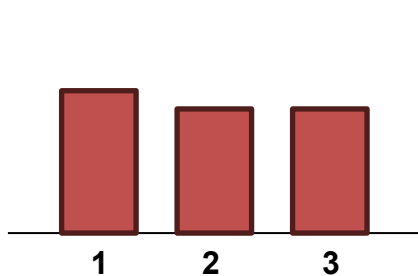
Distill Target

Temperature – $y'(v) = y(v)$, S Confident (Cont.)

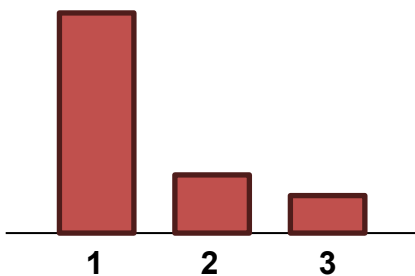
- If the teacher is not confident, it might disturb the student model

True Label : 1

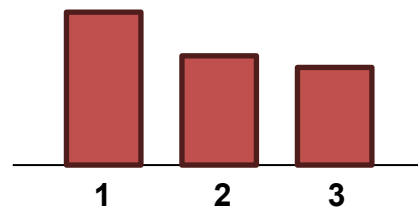
$$\tilde{T}(v) = T_{\max}^{\text{node}} - (T_{\max}^{\text{node}} - T_{\min}^{\text{node}}) \cdot \max(z_v^s)$$
$$p_v^t = \text{softmax}(z_v^t/T), \quad p_v^s = \text{softmax}(z_v^s/T)$$



Teacher Model



Student Model



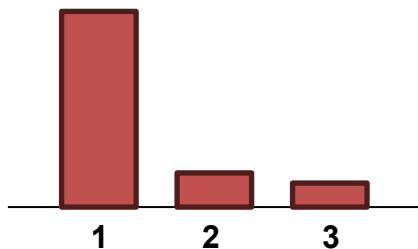
Distill Target

Temperature – $y'(v) = y(v)$, S Low

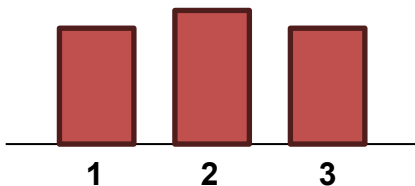
- If true label and the teacher predicted label is equal and the student is not confident with its value, T gets bigger, preserving student more

$$\tilde{T}(v) = T_{\max}^{\text{node}} - (T_{\max}^{\text{node}} - T_{\min}^{\text{node}}) \cdot \max(z_v^s)$$
$$p_v^t = \text{softmax}(z_v^t/T), \quad p_v^s = \text{softmax}(z_v^s/T)$$

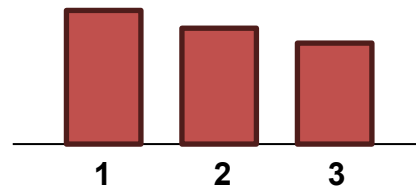
True Label : 1



Teacher Model



Student Model



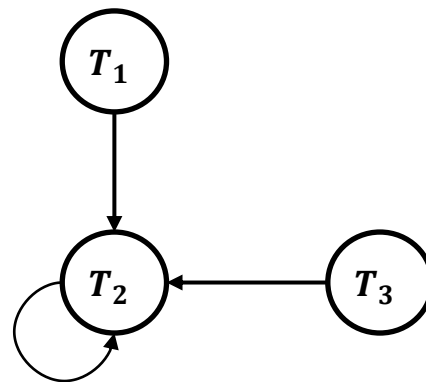
Distill Target

Node-Level Adaptive Distillation

- Aggregate temperatures from neighbors and use KD loss for distillation

$$T(v) = \frac{1}{1+\lambda} \left(\tilde{T}(v) + \lambda \cdot \frac{1}{|\mathcal{N}(v)|} \sum_{u \in \mathcal{N}(v)} \tilde{T}(u) \right)$$

$$\mathcal{L}_{\text{NA}} = \frac{1}{|\mathcal{V}|} \sum_{v \in \mathcal{V}} \text{KL} (p_v^t, p_v^s)$$



Edge-Level Adaptive Distillation

□ Aggregate temperatures from neighbors and use KD loss for distillation

■ $\hat{y}$: predicted probability distribution, I : information entropy, z : soft labels

$$\eta_e = \mathbb{E}_\epsilon \left[\frac{1}{|e|} \sum_{v \in e} (I(\hat{y}_\epsilon(v)) - I(\hat{y}(v)))^2 \right], \quad R_e = 1 - \frac{\eta_e}{\eta_{\max}},$$

$$p(s_i \mid R_{e_i}) \sim \text{Bernoulli}(R_{e_i}), \quad e_i \in \mathcal{E},$$

$$z_e^t = \frac{1}{|e|} \sum_{v \in \mathcal{S}_v(e)} z_v^t, \quad z_e^s = \frac{1}{|e|} \sum_{v \in \mathcal{S}_v(e)} z_v^s, \quad e \in \mathcal{E}'$$

$$\tilde{T}(e) = T_{\max}^{\text{edge}} - (T_{\max}^{\text{edge}} - T_{\min}^{\text{edge}}) \cdot \max(z_e^s),$$

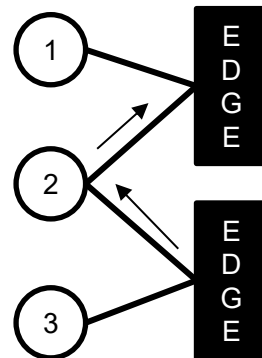
Extract Only the Hyperedges
Stable to Noise

Edge-Level Adaptive Distillation (Cont.)

- Use neighbor hyperedge values that share the same hypernodes

$$T(e) = \frac{1}{1+\gamma} \left(\tilde{T}(e) + \gamma \cdot \frac{1}{|\mathcal{N}(e)|} \sum_{m \in \mathcal{N}(e)} \tilde{T}(m) \right),$$

$$\mathcal{L}_{\text{HA}} = \frac{1}{|\mathcal{E}'|} \sum_{e \in \mathcal{E}'} \text{KL} \left(\text{softmax} \left(\frac{z_e^s}{T(e)} \right), \text{softmax} \left(\frac{z_e^t}{T(e)} \right) \right)$$



Contrastive Loss

□ Embedding distillation while pushing away random node embeddings

■ $V_s(v)$: random negative hypernodes, $\mathcal{E}_s(e)$: random negative hyperedges

$$\mathcal{L}_{\text{NCL}} = - \sum_{v \in \mathcal{V}} \log \frac{\exp(\text{sim}(h_v^s, h_v^t))}{\sum_{v' \in \mathcal{V}_s(v)} \exp(\text{sim}(h_v^s, h_{v'}^t))} \\ - \sum_{v \in \mathcal{V}} \log \frac{\exp(\text{sim}(h_v^s, h_v^s))}{\sum_{v' \in \mathcal{V}_s(v)} \exp(\text{sim}(h_v^s, h_{v'}^s))},$$

Node-Level Contrastive Loss

$$\mathcal{L}_{\text{HCL}} = - \sum_{e \in \mathcal{E}'} \log \frac{\exp(\text{sim}(h_e^s, h_e^t))}{\sum_{e' \in \mathcal{E}'_s(e)} \exp(\text{sim}(h_e^s, h_{e'}^t))} \\ - \sum_{e \in \mathcal{E}'} \log \frac{\exp(\text{sim}(h_e^s, h_e^s))}{\sum_{e' \in \mathcal{E}'_s(e)} \exp(\text{sim}(h_e^s, h_{e'}^s))}.$$

Hyperedge-Level Contrastive Loss

Reinforcement Learning-Based ALC

- Apply strongest weight to the loss with the largest value
 - R : validation accuracy, β : learnable weight, κ : scaling constant

$$r_m \leftarrow (1 - \beta)r_m + \beta R,$$

$$\hat{\alpha}_i^{[t-1]} = \begin{cases} \kappa \cdot \alpha_i^{[t-1]}, & \text{if } i = m \\ \alpha_i^{[t-1]}, & \text{otherwise} \end{cases} \quad \alpha_i^{[t]} = \frac{\hat{\alpha}_i^{[t-1]}}{\sum_{j=1}^5 \hat{\alpha}_j^{[t-1]}}$$

$$\mathcal{L} = \alpha_1^{[t]} \mathcal{L}_{\text{NCL}} + \alpha_2^{[t]} \mathcal{L}_{\text{HCL}} + \alpha_3^{[t]} \mathcal{L}_{\text{NA}} + \alpha_4^{[t]} \mathcal{L}_{\text{HA}} + \alpha_5^{[t]} \mathcal{L}_{\text{CE}}$$

Experiments – Performance

Table 2: Experimental results on graph and hypergraph datasets (%).

Type	Model	Cora	Pubmed	Citeseer	CA-Cora	DBLP-Paper	IMDB-AW
MLP	MLP	49.64±1.13	66.05±2.78	51.69±2.08	51.86±1.47	62.84±1.58	40.87±1.43
GNNs	GCN	79.90±1.75	77.54±1.63	69.58±1.89	72.82±1.70	72.02±1.43	50.62±1.44
	GAT	78.35±2.24	76.54±1.56	69.38±2.33	70.73±1.75	72.53±1.15	49.55±1.82
HGNNs	HGNN	86.64±0.93	84.70±0.68	75.15±1.35	80.51±1.62	83.96±1.42	66.92±0.98
	HGNN ⁺	87.94±0.98	83.82±0.70	74.52±1.01	79.63±1.24	<u>84.13±1.11</u>	66.52±1.40
	UniGCN	<u>88.25±0.86</u>	83.73±0.78	75.42±1.08	80.04±0.65	84.33±0.72	<u>67.31±1.15</u>
GNNs-based distill	GLNN	80.93±1.90	78.36±1.99	69.88±1.66	72.19±3.83	72.50±1.62	50.48±1.51
	KRD	79.47±1.73	78.72±1.94	69.82±3.44	71.75±3.53	72.15±1.69	49.56±2.00
	GraphHI	84.40±0.43	81.92±0.17	73.35±0.31	76.53±0.38	77.12±1.13	54.47±0.69
	NOSMOG	80.12±0.91	80.42±3.00	70.86±3.53	68.96±3.41	71.47±2.36	48.66±1.43
HGNNs-based distill	LightHGNN	84.77±1.18	86.67±0.31	74.79±0.77	77.47±1.90	83.10±1.61	66.96±1.53
	LightHGNN ⁺	85.51±1.74	86.89±0.41	74.52±0.91	79.30±1.84	83.05±1.26	66.87±1.15
	DistillHGNN	79.25±1.37	78.68±1.34	68.18±2.16	68.78±1.16	72.02±1.76	55.27±1.06
	ARCHER (HGNN)	87.37±1.37	<u>87.14±0.91</u>	<u>76.77±0.60</u>	<u>82.42±2.07</u>	83.22±1.44	67.13±1.45
	ARCHER (HGNN ⁺)	88.18±1.10	86.93±0.90	76.50±1.10	82.09±1.98	83.64±1.60	67.10±1.81
	ARCHER (UniGCN)	88.36±1.34	87.49±0.21	77.31±0.60	83.52±0.74	83.96±1.49	68.01±1.38

¹ Best results are in **bold**; second-best are underlined. “ARCHER (·)” denotes our model implemented with different HGNN teacher backbones.

Experiments – Inference Time

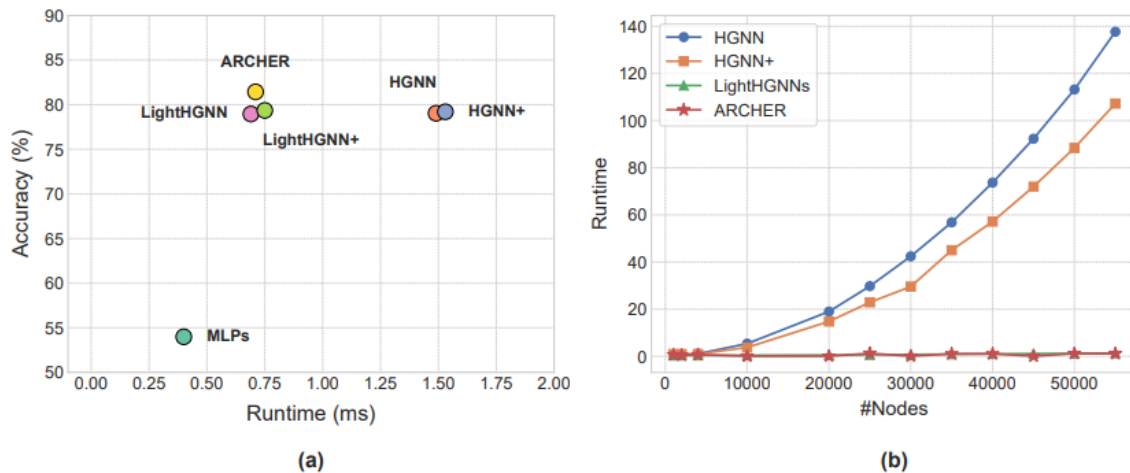


Figure 3: (a) Average runtime and accuracy across six datasets; (b) runtime comparison on synthetic hypergraph datasets.

Experiments – Node Embedding

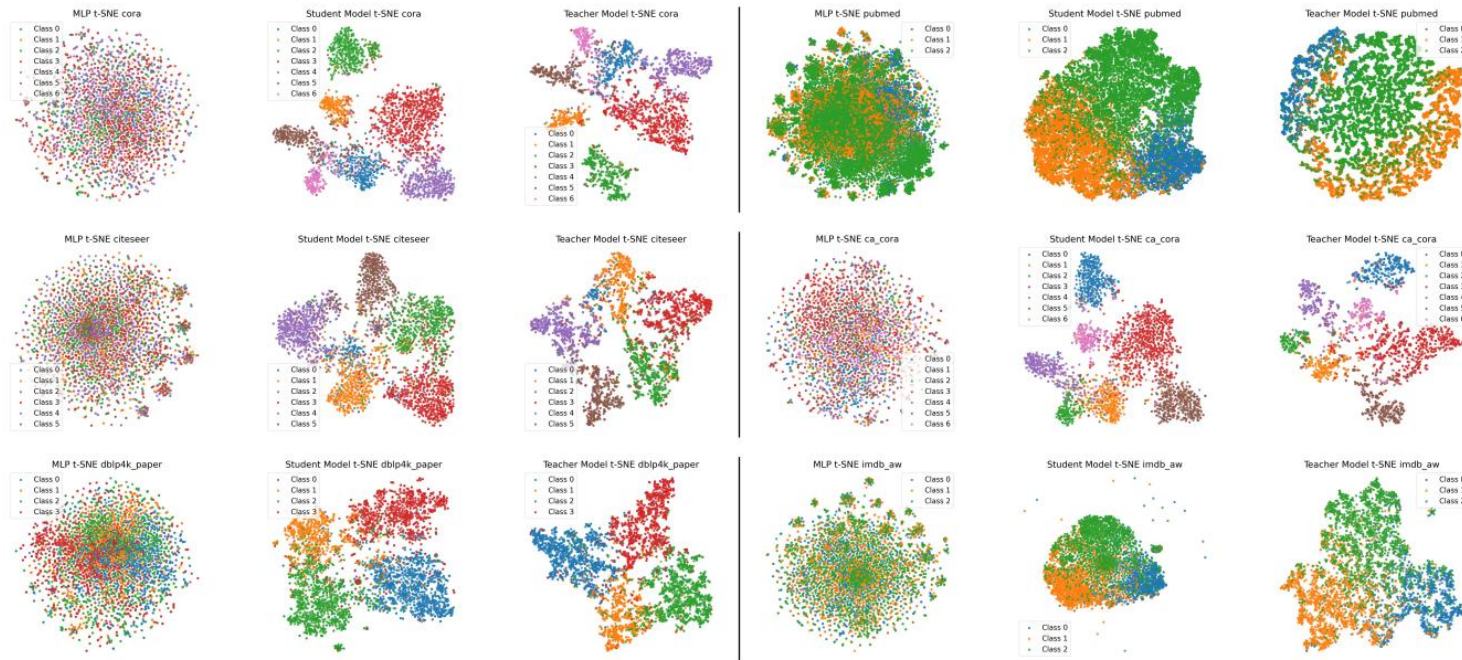


Figure 4: Node embedding visualization. Each dataset (3 images) is grouped and separated by a vertical line.

□ Contribution

- First hypergraph distillation model to use adaptive learning
- Attempts to use student model results as well as the teacher model

□ Limitation

- Role of temperature functions is unclear
- Contrastive learning works similar to simple embedding distillation with some regularizations
- Reinforcement learning method does not have a strong role



Hypergraph-MLP : Learning on Hypergraphs Without Message Passing

ICASSP '24

Bohan Tang¹, Siheng Chen^{2,3}, Xiaowen Dong¹
University of Oxford¹, Shanghai Jiao Tong University²,
Shanghai AI Laboratory³

Smoothing of Hypergraphs

- In hypergraphs, propagations and the nonlinear function following it reduces high frequency energy, resulting in smoothing

$$\mathbf{X}^{(\ell+1)} = \sigma(\mathbf{F}\mathbf{X}^{(\ell)}\mathbf{\Theta}^{(\ell)}), \quad \mathbf{F} := \mathbf{D}_v^{-1/2}\mathbf{H}\mathbf{D}_e^{-1}\mathbf{H}^\top\mathbf{D}_v^{-1/2}$$

$$\|\mathbf{P}\sigma(x)\|_2 \leq \|\mathbf{P}x\|_2.$$

$$\begin{aligned} \sum_{c=1}^C \left\| \mathbf{P}\mathbf{X}_c^{(\ell)} \right\|_2 &\leq \sum_{c=1}^C \left\| \mathbf{P}\mathbf{F}\mathbf{X}_c^{(\ell-1)} \right\|_2 \\ &\leq \lambda_{\max} \left\| \mathbf{P}\mathbf{X}^{(\ell-1)} \right\|_{Fro} \end{aligned}$$

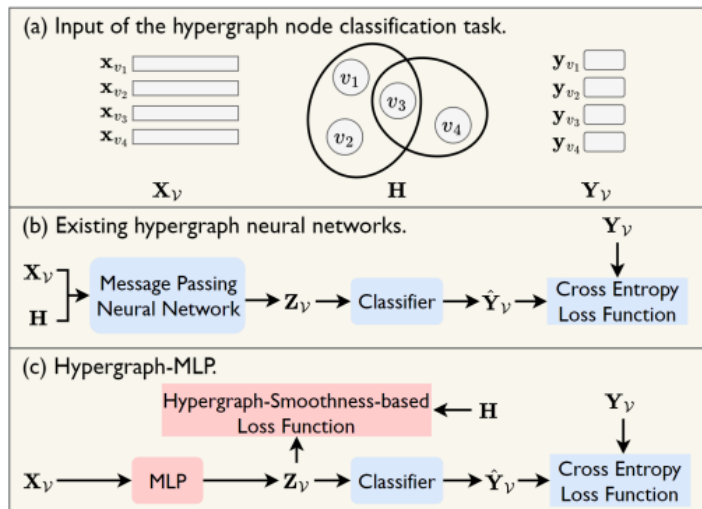
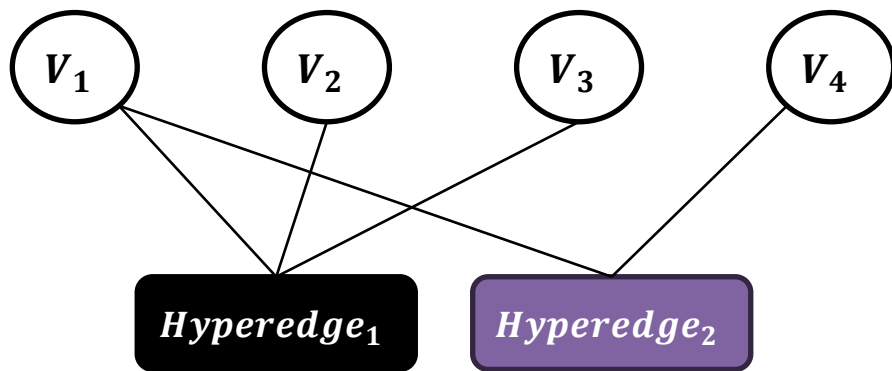


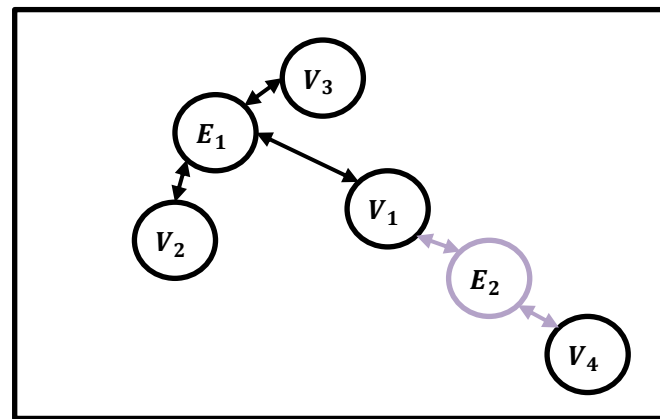
Fig. 1: The training paradigm of the Hypergraph-MLP and hypergraph neural networks for hypergraph node classification, where $\mathbf{X}_V$ denotes node features, $\mathbf{H}$ is the incidence matrix, $\mathbf{Y}_V$ denotes node labels, and only partial labels in $\mathbf{Y}_V$ are used at training.

Star Expansion

- Representing hypergraphs as graphs by treating hyperedges as auxiliary nodes



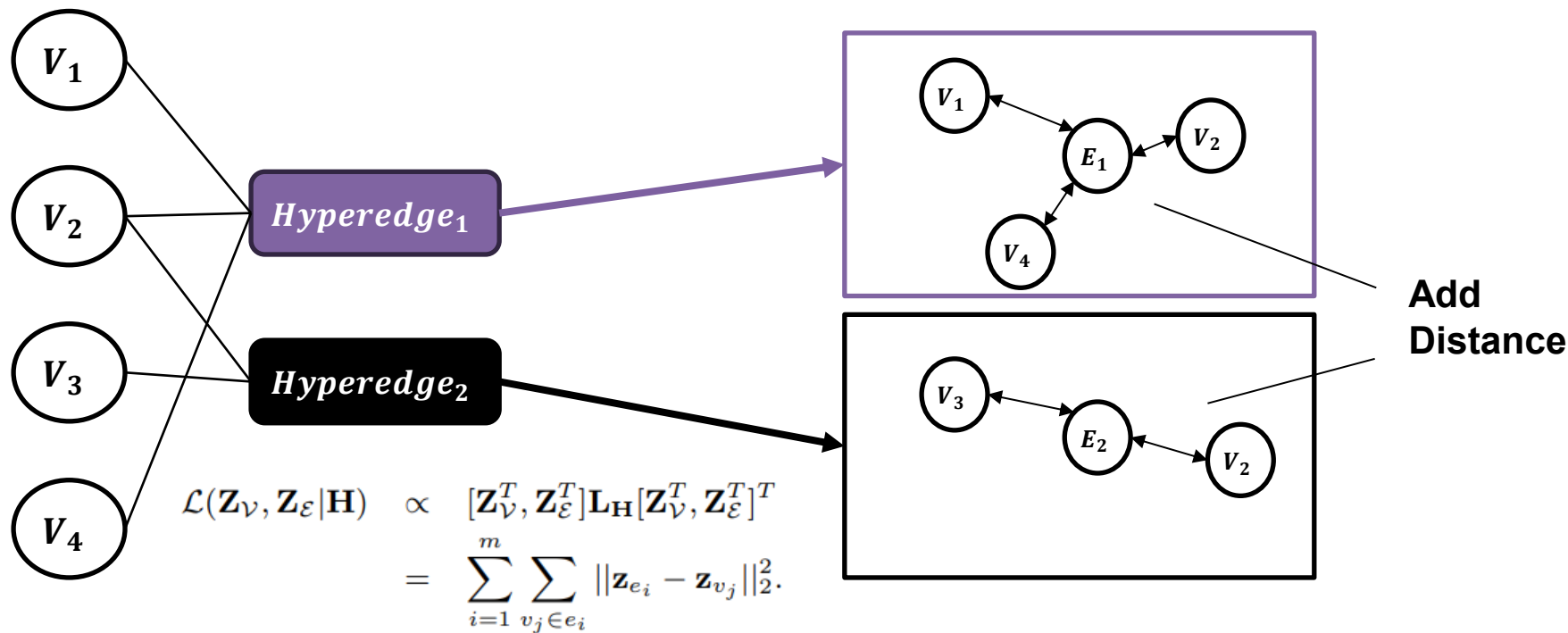
Hypergraph



Embeddings

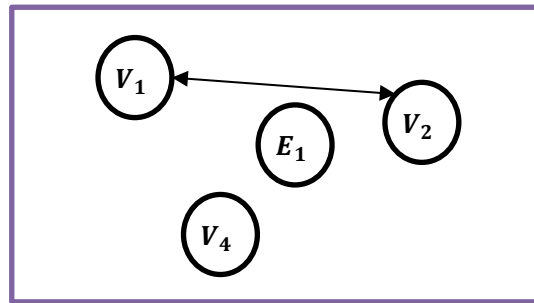
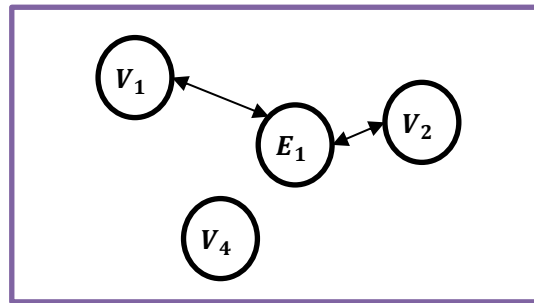
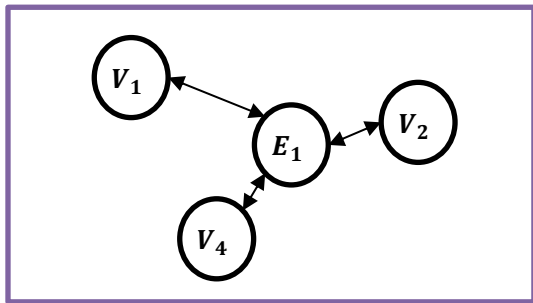
Smoothness Prior

□ If the MLP follows how hypergraphs smoothen, it can imitate the role



Approximation

□ As MLP does not receive edge information, approximation needed



$$\sum_{i=1}^m \sum_{v_j \in e_i} \|\mathbf{z}_{e_i} - \mathbf{z}_{v_j}\|_2^2 \geq \sum_{i=1}^m (\|\mathbf{z}_{e_i} - \mathbf{z}_{v_{a_i}}\|_2^2 + \|\mathbf{z}_{e_i} - \mathbf{z}_{v_{b_i}}\|_2^2) \geq \sum_{i=1}^m \max_{v_j, v_k \in e_i} (\|\mathbf{z}_{v_j} - \mathbf{z}_{v_k}\|_2^2)$$

$$\Rightarrow \ell_{smooth} = \frac{1}{m} \sum_{i=1}^m \max_{v_j, v_k \in e_i} (\|\mathbf{z}_{v_j} - \mathbf{z}_{v_k}\|_2^2)$$

Experiment – Performance

Table 2: Comparison with baselines on clean datasets. Mean testing **ACC (%)** $\pm$ standard deviation from 20 runs.

	Cora	Citeseer	Pubmed	DBLP	20News	NTU2012	House	Avg Mean
HyperGCN	78.45 $\pm$ 1.26	71.28 $\pm$ 0.82	82.84 $\pm$ 8.67	89.38 $\pm$ 0.25	81.05 $\pm$ 0.59	56.36 $\pm$ 4.86	78.22 $\pm$ 2.46	76.80
HGNN	79.39 $\pm$ 1.36	72.45 $\pm$ 1.16	86.44 $\pm$ 0.44	91.03 $\pm$ 0.20	80.33 $\pm$ 0.42	87.72 $\pm$ 1.35	66.16 $\pm$ 1.80	80.50
HCHA	79.14 $\pm$ 1.02	72.42 $\pm$ 1.42	86.41 $\pm$ 0.36	90.92 $\pm$ 0.22	80.33 $\pm$ 0.80	87.48 $\pm$ 1.87	67.91 $\pm$ 2.26	80.66
UniGCNII	78.81 $\pm$ 1.05	73.05 $\pm$ 2.21	88.25 $\pm$ 0.40	91.69 $\pm$ 0.19	81.12 $\pm$ 0.67	89.30 $\pm$ 1.33	80.65 $\pm$ 1.96	83.27
AllDeepSets	76.88 $\pm$ 1.80	70.83 $\pm$ 1.63	88.75 $\pm$ 0.33	91.27 $\pm$ 0.27	81.06 $\pm$ 0.54	88.09 $\pm$ 1.52	80.70 $\pm$ 1.59	82.51
AllSetTransformer	78.59 $\pm$ 1.47	73.08 $\pm$ 1.20	88.72 $\pm$ 0.37	91.53 $\pm$ 0.23	81.38 $\pm$ 0.58	88.69 $\pm$ 1.24	83.14 $\pm$ 1.92	83.59
MLP	74.99 $\pm$ 1.49	72.31 $\pm$ 1.28	87.69 $\pm$ 0.59	85.53 $\pm$ 0.27	81.70 $\pm$ 0.49	87.89 $\pm$ 1.36	83.78 $\pm$ 1.96	81.98
Hypergraph-MLP	79.80 $\pm$ 1.82	73.90 $\pm$ 1.57	87.89 $\pm$ 0.55	90.29 $\pm$ 0.26	81.75 $\pm$ 0.41	88.42 $\pm$ 1.32	84.03 $\pm$ 1.75	83.72

Experiment – Inference Time

Table 3: Comparison with baselines on clean datasets. Mean **inference time** (ms) $\pm$ standard deviation from 60000 runs.

	Cora	Citeseer	Pubmed	DBLP	20News	NTU2012	House	Avg Mean
HyperGCN	0.46 ± 0.10	0.49 ± 0.10	0.60 ± 0.07	1.19 ± 0.21	0.82 ± 0.30	0.47 ± 0.10	0.50 ± 0.10	0.65
HGNN	1.40 ± 0.11	1.41 ± 0.33	3.33 ± 0.26	4.36 ± 0.41	1.39 ± 0.33	1.26 ± 0.11	1.43 ± 0.14	2.08
HCHA	1.33 ± 0.38	1.37 ± 0.14	3.43 ± 0.17	8.08 ± 0.76	1.45 ± 0.14	1.29 ± 0.15	1.46 ± 0.16	2.63
UniGCNII	3.01 ± 0.14	0.71 ± 0.10	0.85 ± 0.07	21.15 ± 0.64	3.79 ± 0.18	1.48 ± 0.08	0.77 ± 0.13	4.54
AllDeepSets	2.02 ± 0.15	2.40 ± 0.14	8.90 ± 1.09	21.51 ± 0.43	7.68 ± 0.57	1.96 ± 0.09	1.39 ± 0.13	6.55
AllSetTransformer	1.91 ± 1.34	3.01 ± 1.32	4.75 ± 1.33	21.21 ± 1.42	4.30 ± 1.57	1.90 ± 1.32	2.14 ± 1.29	5.60
Hypergraph-MLP	0.45 ± 0.11	0.21 ± 0.07	0.19 ± 0.04	0.47 ± 0.17	0.28 ± 0.05	0.36 ± 0.08	0.28 ± 0.05	0.32

☐ Contribution

- Showed that MLP can solely perform well without hypergraph neural networks
- Experimentally showed that simply aggregating nodes in the same hyperedges can imitate hypergraphs

☐ Limitation

- Smoothing can still happen while aggregation in the embedding space
- Only limited to transductive learning